Python Learning Roadmap

Stage 1: Beginner

Objective: Understand Python basics and develop foundational programming skills.

Timeline: 5-7 Days

1. **Python Basics:**
    - Install Python and set up an IDE (VS Code, PyCharm, or Jupyter).
    - Create Virtual environment & managing package with pip.
    - Handling of requirements.txt file.
    - Data Types, Variables, and Operators.
    - Input/Output Operations
    - Control Flow (if-else, loops).
    - Functions & *Args/**Kwargs and Scope.
    - Inbuilt Function & Lambda Function
    - Break, Continue, Pass keyword uses & range() Function
2. **Core Data Structures:**
    - Lists, Tuples, Sets, and Dictionaries.
    - Comprehensions (List, Dictionary).
    - String Manipulation & formatting.
3. **Modules and Packages:**
    - Using built-in modules (math, random, os, datetime).
    - Creating and importing custom modules.
    - Regex library.
4. **Error Handling:**
    - Try-Except Blocks.
    - Raising Exceptions.
5. **Basic File Handling:**
    - Reading and writing files.
    - Working with CSV files.
6. **First Projects:**
    - Simple CLI Calculator.
    - To-Do List Application.(No need to develop UI App)

Stage 2: Intermediate

Objective: Learn Python's advanced features, focus on writing modular code, and prepare for real-world applications.

Timeline: 5 Days

1. **Object-Oriented Programming (OOP):**
    - Classes and Objects.
    - Inheritance, Polymorphism, Encapsulation.
    - Mixins, Multiple Inheritance.
    - Special/Magic Methods (`__init__`, `__str__`, etc.).
    - Access Modifiers & Escape character
2. **Python Standard Library:**
    - Collections (Counter, defaultdict, deque).
    - itertools (combinations, permutations, etc.).
    - functools (partial, reduce, lru_cache).
3. **Advanced File Operations**
    - JSON and XML Parsing.
    - Config File Handling (INI, YAML).
4. **Working with Databases:(Optional)**
    - Introduction to SQL.
    - Using SQLite with Python.
5. **Web Scraping:**
    - Introduction to `requests` and (Optional)`BeautifulSoup`.
    - Automating tasks with `selenium`. (Optional)
6. **Unit Testing:**
    - Using `unittest` and `pytest`.
    - Mocking and patching.
    - Setup Debugger in project.

Stage 3: Advanced Python

Objective: Dive into Python's advanced capabilities and learn concepts crucial for scalable backend development.

Timeline: 7 Days

1. **Advanced Python Concepts:**
    - Decorators and Context Managers.
    - AsyncIO for asynchronous programming.
    - Generators and Coroutines
    - Multithreading vs Multiprocessing.
2. **Data Handling:**
    - Pandas for Data Analysis.
    - Working with APIs (Requests, JSON, RESTful APIs).
3. **Logging and Debugging:**
    - Using `logging` module effectively.
    - Debugging with a Python debugger.
4. **Practical Task:**
    - Implement a small demo demonstrating usage of pandas, take any dataset or csv of your choice.

Stage 4: Django Basics

Objective: Get started with Django for building robust web applications.

Timeline: 7 Days

1. **Django Fundamentals:**
    - Installing Django and setting up projects.
    - MVT (Model-View-Template) architecture.
    - Class based and functional based approach.
    - Purpose of settings.py, manage.py and urls.py
    - Models, Views, and Templates Basics.
    - URL Routing and Static Files.
    - Test cases.
2. **Database Handling:**
    - Setting up Django ORM.
    - Querysets, Managers, and Migrations.
    - Relationships(one-to-one, many-to-one, many-to-many)
    - Model methods.
3. **Forms and Validations:**
    - Handling Forms and CSRF.
    - Validating User Inputs.
4. **Authentication and Authorization:**
    - User Authentication System.
    - Override default user model.
    - Role-Based Access Control.
5. **Django Project:**
    - Build a demo project: **Python practical task**

Stage 5: Advanced Django and DRF

Objective: Master Django and DRF for building scalable, production-grade APIs.

Timeline: 8-10 Days

1. **Django Rest Framework (DRF):**
    - Serializers and Views.
    - Viewsets.
    - Routers and Endpoints.
    - Authentication in DRF (Token, JWT, OAuth2).
2. **Advanced Django Features:**
    - Middleware and Custom Decorators.
    - Signals and Caching.
    - File Upload and Management.
3. **Database Optimization:**
    - Query Optimization (select_related, prefetch_related).
    - Indexing and Query Planning.

4. **Scalability and Performance:**
    - Asynchronous Django (ASGI and Channels).
    - Rate Limiting and Throttling in DRF.
    - Pagination Handling
    - API Versioning.
5. **DevOps and Deployment:**
    - Setting up a CI/CD Pipeline.(optional)
    - Dockerizing Django Applications as per production environment.
    - Deployment with Gunicorn, Nginx, and AWS.(optional)
6. **Django Project:**
    - Blog application. - Build an application combining concepts of the above stages.
    - Additionally implement below cases:
        i. While adding blogs add a functionality to get blog summary from chat GPT and store it in Database, utilize django receiver signal.
        ii. If a user does not pass tags while creating a blog, use the ChatGPT API to tag those blogs and store the result.

Stage 6: Mastering Backend Engineering(Optional)

Objective: Build a solid understanding of backend architecture and implement best practices.

1. **Backend Architecture:**
    - Microservices vs Monolithic Architecture.
    - Designing Scalable Systems.
    - Event-Driven Programming.
2. **Caching Strategies:**
    - Using Redis and Memcached with Django.
    - Distributed Caching.
3. **Advanced Security:**
    - Securing APIs with HTTPS and Authentication.
    - Handling SQL Injection and XSS.
4. **Logging and Monitoring:**
    - Setting up Monitoring with Prometheus and Grafana.
    - Centralized Logging Systems.
5. **Final Projects:**
    - Scalable Multi-Tenant E-commerce Platform.
    - Social Media Platform with Real-Time Features.

General Tips for Success

1. **Practice Daily:**
    - Solve coding challenges on platforms like LeetCode, HackerRank, or Codewars.
    - Build small projects to apply concepts learned.
2. **Read Documentation:**
    - Familiarize yourself with Django and DRF's official documentation.
3. **Follow Best Practices:**
    - Adhere to PEP-8 for Python code.
    - Use Git and GitHub for version control.
4. **Explore Beyond:**
    - Learn additional tools like Celery for task queues.

Helpful Links:

Learn by topics: https://roadmap.sh/python

Django framework: https://docs.djangoproject.com/en/5.1/

Django rest framework: https://www.django-rest-framework.org/

Pandas Tutorial: https://www.youtube.com/watch?v=2uvysYbKdjM

For stage 1 to stage 3, you can refer this youtube channel:

- Core Concept python reference videos on https://www.youtube.com/@mCoding

Django Playlist that I used personally (jeel) - https://www.youtube.com/playlist?list=PL-osiE80TeTtoQCKZ03TU5fNfx2UY6U4p

Both **mCoding** and **Corey Schafer** provide videos for concepts as well as library specific playlists like pandas.